

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..45

REDIS

 Copy as Markdown

Redis

Turn Redis on when more than one Serverpod instance must share a cache, pass messages, or revoke authentication together. Redis is optional. Without it, those features stay local to each process, so a second instance does not see the first instance's cache or events. This page covers enabling Redis, the password, the development Docker instance, and connecting a client such as RedisInsight. For using the cache and messages once Redis is up, see [Caching](#) and [Server events](#).

Whether a new project includes Redis configuration depends on how you created it:

- `serverpod create` **with the setup screen**: only if you select Redis.
- `serverpod create` **without the setup screen** (for example with `--no-interactive`): included by default.
- `serverpod quickstart`: always included.

Including it copies the config, the Compose service, and a generated password. It doesn't turn Redis on.

Connection details

Each run-mode file can contain a `redis` block. For local development, that file is `config/development.yaml`.

config/development.yaml

```
redis:
  enabled: false
  host: localhost
  port: 8091
  # user:           # defaults to empty
  # requireSsl: true # defaults to false
```



The `host` is the domain name or IP address of your Redis instance, and `port` is the port Redis is listening on. The `user` and `requireSsl` keys are optional. Leave `user` unset unless your Redis server uses ACL usernames. The `requireSsl` key defaults to `false`.

The scaffold writes `enabled: false` in every run mode. If the `redis` block exists and you omit `enabled`, Serverpod treats Redis as enabled. When Redis is disabled, Serverpod does not load it, and the password is optional.

File	<code>enabled</code>	Host	Port
<code>config/development.yaml</code>	<code>false</code>	<code>localhost</code>	8091
<code>config/test.yaml</code>	<code>false</code>	<code>localhost</code>	9091
<code>config/staging.yaml</code>	<code>false</code>	<code>redis.private-staging.examplepod.com</code>	6379
<code>config/production.yaml</code>	<code>false</code>	<code>redis.private-production.examplepod.com</code>	6379

Development and test use 8091 and 9091 so they do not clash with a Redis installed on the machine's default port 6379. The staging and production templates use 6379 and placeholder private hostnames you replace with your own.

If your project has no `redis` block, add the snippet above, a password (see [Redis password](#)), and a Redis service to `docker-compose.yaml`.

The `host`, `port`, `user`, `enabled`, and `requireSsl` keys can each be set with environment variables instead of YAML. See the [configuration reference](#) for the full list. You can enable Redis with environment variables alone, with no `redis` block in YAML, if host, port, and password are set.

Redis password

The Redis password is stored in `config/passwords.yaml` under the run-mode key, as a sibling of `database`. It is not nested under `database`.

```
config/passwords.yaml
```

```
development:
  database: '<database password>'
  redis: '<the generated password>'
```

The create command generates a password for `development` and `test` only. Staging and production templates do not include a `redis` password. Add one under that run mode before you enable Redis there, or set it through an environment variable.

```
export SERVERPOD_PASSWORD_redis='...'
```

```
export SERVERPOD_REDIS_PASSWORD='...'
```

Both variables set the same secret. If both are set, `SERVERPOD_PASSWORD_redis` wins. Environment variables override the passwords file. See [Manage secrets](#) for how secrets are loaded.

If Redis is enabled and the password is missing, the server refuses to start:

```
Missing password for "redis". Please check your config/passwords.yaml file or the
`SERVERPOD_PASSWORD_redis` environment variable.
```

Enable Redis

Set `enabled` to `true` in the run-mode file you will start:

```
config/development.yaml
```

```
redis:
  enabled: true
  host: localhost
  port: 8091
```

When Redis is enabled, `host`, `port`, and a password are required.

The development Redis

The scaffolded development Redis is a Docker container from the server package's `docker-compose.yaml`. It uses the `redis:6.2.6` image, publishes host port `8091` to container port `6379`, and starts Redis with password-only authentication (`--requirepass`). There is no username.

New projects use an [embedded PostgreSQL](#) (`database.dataPath`), so `serverpod start` does not bring Compose up on its own. Start Redis yourself from the server package:

```
docker compose up --detach
```

To start Compose whenever the server starts, pass `--docker`:

```
serverpod start --docker
```

See [Running your server](#) for when Compose is started automatically.

To stop Redis:

```
docker compose stop
```

To remove the containers and **delete** all associated data:

```
docker compose down -v
```

Connect a GUI

Use these values from your machine for RedisInsight, `redis-cli`, or any other Redis client.

CAUTION

Redis itself listens on port 6379. Serverpod's development setup publishes **8091** on the host to avoid conflicts. A GUI running on your machine must use 8091, not 6379.

Development (Docker from the template)

Field	Value
Host	<code>localhost</code>
Port	<code>8091</code> (not 6379)
Username	empty

Field	Value
Password	<code>development.redis</code> in <code>config/passwords.yaml</code>
TLS	off (<code>requireSsl: false</code>)

```
redis-cli -h localhost -p 8091 -a '<password from passwords.yaml>'
```

Test (Docker from the template)

Field	Value
Host	<code>localhost</code>
Port	<code>9091</code>
Username	empty
Password	<code>test.redis</code> in <code>config/passwords.yaml</code>

The default Docker Redis authenticates with a password only. Leave the username empty. If you set `user` in YAML, Serverpod sends Redis ACL authentication with that username and the password.

Inside the Compose network, Redis listens on **6379** with the same password. Port 8091 is only the mapping published to the host.

Test, staging, and production

The test instance is a second Compose service on host port **9091**, with its password under `test.redis` in `passwords.yaml`. Enable it in `config/test.yaml` the same way as development.

Staging and production templates listen on **6379** and ship with placeholder hosts. They do not include a generated Redis password. Before you set `enabled: true` in those files:

1. Point `host` at your Redis instance.
2. Add a `redis` password under that run mode in `passwords.yaml`, or set `SERVERPOD_PASSWORD_redis` / `SERVERPOD_REDIS_PASSWORD`.
3. Set `requireSsl` and `user` if your provider requires them.

On [Serverpod Cloud](#), set the same values with `serverpod cloud password set` and `serverpod cloud variable set`. See [Use Redis for PubSub and caching](#).

SSL and username

Managed Redis (Upstash, ElastiCache, Memorystore, and similar) often requires TLS, and some providers require a username.

config/production.yaml

```
redis:
  enabled: true
  host: your-redis-host.example.com
  port: 6379
  user: default
  requireSsl: true
```

The `requireSsl` key defaults to `false`. Set it to `true` when the provider requires TLS. Set `user` only when the provider uses Redis ACL usernames. The default Docker Redis in development has neither.

For Serverpod Cloud with a third-party Redis such as Upstash, follow [Use Redis for PubSub and caching](#).

What Redis is used for

Local caches (`session.caches.local`, `localPrio`, and `query`) never use Redis.

Feature	With Redis	Without Redis
<code>session.caches.global</code>	Shared across instances	Isolated in-memory fallback, not shared
<code>session.messages</code> (default)	Pub/sub across the cluster	Local to this process
<code>MessageScope.global</code>	Cluster-wide	Throws <code>StateError</code>
Auth revocation across servers	Propagated through Redis	Posted locally only
<code>/readyz</code>	Includes a Redis connection check	Redis check omitted

The in-memory fallback for `session.caches.global` is a separate cache, not `local` or `localPrio`. See [The global cache and Redis](#).

Cluster messaging is best effort. Failed publishes are not retried. The `MessageScope.global` scope requires Redis and throws `StateError('Redis needs to be enabled to use this method')` without it. See [Message scope](#).

The `/readyz` probe checks Redis only when Redis is enabled. The `/livez` probe does not.

In every run mode except production, a failed Redis connection at startup logs that Serverpod is falling back to a local cache, and the server still starts. In production there is no fallback: the global cache stays Redis-backed even if Redis is down.

To send Redis commands beyond the cache and pub/sub APIs, borrow the connection Serverpod manages. See [Send Redis commands directly](#).

Troubleshooting

Redis is in my project but nothing connects

The templates write `enabled: false`. Opting into Redis at create only includes the config, Compose service, and password. Set `enabled: true` in the run-mode file, then start Redis.

A GUI or `redis-cli` cannot authenticate

The password is `development.redis` in `config/passwords.yaml`, not nested under `database`. The default Docker Redis has no username: leave that field empty. From the host, connect to port **8091**, not 6379.

The server started but instances do not share state

In every run mode except production, a down Redis does not stop the server. Confirm the container is running and that `/readyz` reports a passing Redis check. Production does not fall back.

Staging or production fails at startup with a missing Redis password

Those templates do not generate a `redis` password. Add `redis:` under that run mode in `passwords.yaml`, or set `SERVERPOD_PASSWORD_redis` / `SERVERPOD_REDIS_PASSWORD`, before enabling Redis.

Related

- [Configuration](#): run modes, YAML files, and secrets.
- [Configuration reference](#): every Redis option and environment variable.
- [Caching](#): the global cache Redis backs.

- [Server events](#): cluster messaging through Redis.
- [Use Redis for PubSub and caching](#): managed Redis on Serverpod Cloud.